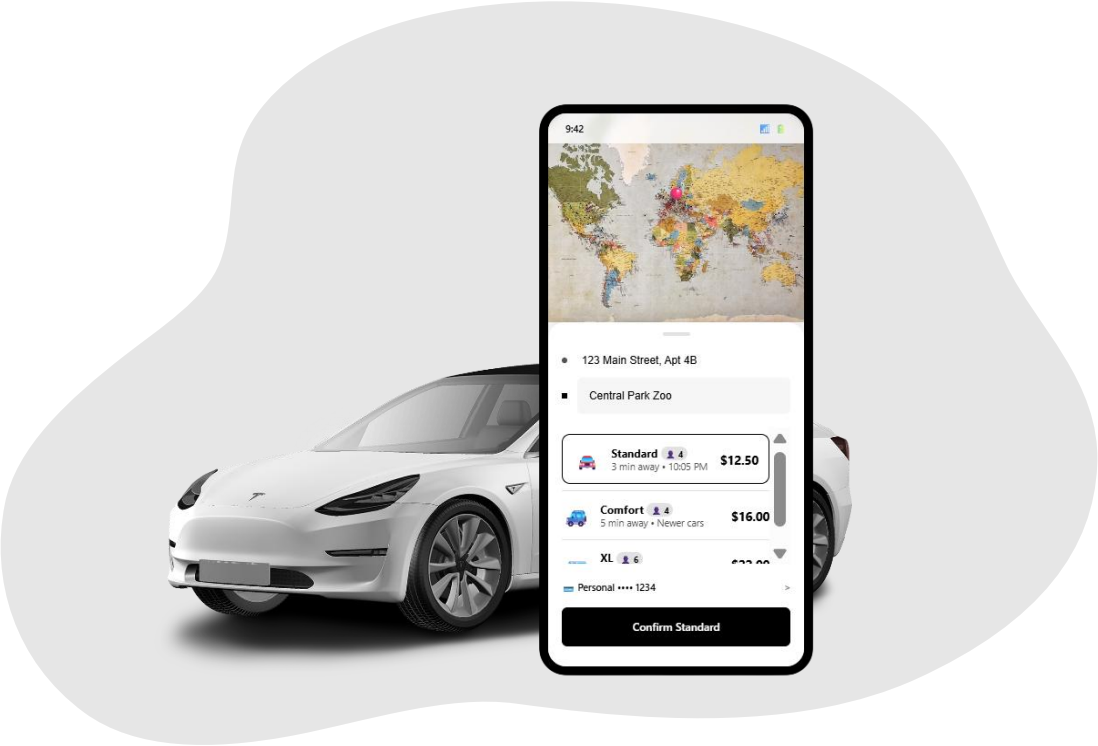
香港城市大學
City University of Hong Kong

CS3342 Software Design

2025-26 Semester B



Ride-hailing System Report

59300549	Pan Qi Yuan	59300863	Chan Shing Shun
59299614	Heung Chi Ho	59295793	Chan Kin Ching
59302402	Lee Kwan Shing	59302358	Fan Wai Tsun

Date: 20/04/2026

Table of Contents

1. Introduction & Background 4

1.1 Project Background & Motivation 4

1.2 Project Objectives / Goals 4

1.3 Features & Functionality 5

1.4 Project Scope 6

1.5 Risks, Constraints & Feasibility 6

2. System Requirements 7

2.1 Stakeholders & Users 7

2.2 Functional Requirements 8

2.3 Non-functional Requirements 9

2.4 Use Case Diagram 11

2.5 Use Case Specification 11

3. System Design 20

3.1 Class Diagram 20

3.2 Description of Class Diagram 21

3.3 Sequence Diagrams / Object Interactions 22

4. Utilization of Software Design Principles and Patterns 27

4.1 Applied Design Principles 27

4.2 Applied Design Patterns 29

5. System Prototype 36

5.1 UI Design & User Flow 36

6. Project Management 37

6.1 Lifecycle Management and Development Process 37

6.1 Team Roles & Work Allocation 38

6.2 Weekly Log 38

6.3 Risk Management 39

7. Conclusion & Reflection 40

8. References 41

1. Introduction & Background

1.1 Project Background & Motivation

The urban transportation industry has been significantly transformed by the advent of ride-hailing platforms like Uber and DiDi. Traditional transportation systems often struggle with inefficient dispatching, lack of pricing transparency, and high empty-cruising rates for drivers. Motivated by these industry challenges, this project aims to design and implement a comprehensive Ride-hailing System backend using Java. The primary motivation is to practically apply Object-Oriented Programming (OOP) concepts and UML modeling to bridge abstract software requirements with a concrete, logical architecture. By leveraging classic software design patterns, we intend to simulate a fully functional ride lifecycle—from the initial user request to the final payment processing—demonstrating a robust, loosely coupled, and highly cohesive system. This project focuses on backend architectural design rather than real-time GPS accuracy, allowing deeper exploration of UML modeling and OOP principles.

1.2 Project Objectives / Goals

The core objective is to deliver a functional backend prototype and a rigorous design architecture for a Ride-hailing System These objectives are addressed in Sections 2 to 4 through requirements analysis, system design, and design pattern application. Specific goals include:

- **Architectural Design:** Utilizing formal UML modeling to define clear boundaries and interactions among system entities. This includes a comprehensive Class Diagram and detailed Sequence Diagrams for critical operations such as **Registration & Setup, Ride Requests, and Driver Workflows**.
- **Design Pattern Integration:** Applying structural and behavioral patterns (e.g., Singleton, Factory, State, Strategy, and Observer) to resolve architectural complexities and strictly adhere to SOLID principles.

- **Efficient Dispatching:** Developing a simulated mapping service to compute routes and accurately pair passengers with nearby, eligible drivers.
- **Comprehensive Lifecycle Simulation:** Implementing a reliable state machine that safely transitions a ride request through all legal phases without logical conflicts.
- These objectives are addressed respectively in Sections 2–4 through requirements analysis, UML modeling, and pattern application.

1.3 Features & Functionality

The backend system is divided into several highly cohesive modules, directly reflecting our system models, to deliver a seamless ride-hailing experience:

- **Authentication & User Management:** Handles secure registration, setup, and login for "PASSENGER" and "DRIVER" roles via a Factory pattern. Passwords are secured using BCrypt hashing. Drivers can manage specific vehicle types (e.g., Standard or SUV) and toggle their online availability.
- **Map & Dispatching Module:** Manages online drivers and passenger requests. It relies on a `DummyMapProvider` that applies the Haversine formula to calculate the straight-line spherical distance between locations and estimates the time of arrival (ETA) based on an average city speed of 30 km/h.
- **Real-time Notifications (Observer Pattern):** When a ride is generated, the system uses the Observer pattern to broadcast push notifications to nearby online drivers, allowing them to view the estimated fare and distance before accepting the assignment.
- **Ride State Management (State Pattern):** The lifecycle of a ride is strictly controlled. Transitions flow sequentially from `Requested` → `Accepted` → `InTransit` → `Completed` → `Paid`, actively preventing illegal actions such as starting an already completed ride.
- **Dynamic Pricing (Strategy Pattern):** The system calculates fares using interchangeable algorithms, supporting `StandardPricing` (e.g., a base multiplier of 1.5x/km) and `SurgePricing` (e.g., a multiplier of 3.0x/km for peak hours).

- **Payment Processing:** Upon mutual arrival confirmation, the system calculates the final fare, deducts a platform commission (e.g., 20%), and credits the remaining balance to the driver. It enforces a "payable" state check to prevent duplicate charges.

1.4 Project Scope

To ensure timely delivery, the project boundaries are strictly defined:

- **In-Scope:** Object-oriented backend implementation in Java, comprehensive UML diagram generation (Use Case, Class, and Sequence diagrams for core flows), state machine enforcement, dummy map calculations, dynamic pricing strategies, simulated payment processing, and static UI mockups.
- **Out-of-Scope:** Integration with live external mapping APIs (e.g., live Google Maps), real third-party payment gateways (e.g., Stripe, PayPal), real-time GPS tracking hardware, and fully integrated frontend client applications (iOS/Android). However, because our architecture strictly adheres to SOLID principles, the system is highly decoupled. This means we can easily integrate these real-world external features later on if we decide to expand the project into a fully functional product.

1.5 Risks, Constraints & Feasibility

- **Technical Feasibility:** The project is highly feasible. Developing the core backend in Java alongside a simulated map provider allows the team to thoroughly test dispatching and design patterns without relying on paid, rate-limited external APIs.

- **Constraints:** The project is constrained by a strict development timeline, requiring precise coordination between the design phase (diagram generation) and the coding phase.

- **Risks & Mitigation:**

  Risk: Logical deadlocks or illegal operations during complex workflows (e.g., driver accepting a canceled ride).

  Mitigation: Strong implementation of the State Design Pattern alongside rigorous pre-coding Sequence Diagrams to map out and agree upon all object interactions before execution.

  Risk: Route calculation inaccuracies due to the lack of live maps.

  Mitigation: Adopting the standard Haversine formula to approximate real-world spherical distances reliably, which provides sufficient mathematical accuracy for a backend proof-of-concept.

2. System Requirements

2.1 Stakeholders & Users

In the context of the Ride-hailing System backend prototype, the stakeholders and users can be categorized into direct system users, external service entities, and project stakeholders.

1. Primary Users (Direct System Actors)

- **Passengers:** These are the primary consumers of the platform who register accounts to request transportation services. Their core interactions include inputting pickup and drop-off locations, selecting acceptable vehicle types (e.g., standard cars or SUVs), viewing estimated times of arrival (ETAs) and fares, and processing payments securely at the end of the trip.
- **Drivers:** Individuals who register their vehicles and toggle their online availability to provide transportation services. They act as active participants who receive real-time push notifications for nearby ride requests, accept orders, navigate to destinations, and earn income. They receive 80% of the generated fare for successfully completed rides.

- **Customer Service / Platform Administrators:** Support staff responsible for monitoring system integrity and resolving trip irregularities. For instance, if a completed route severely deviates from the initial estimated distance, customer service intervenes to conduct a manual review before the final payment is processed.

2. Secondary Stakeholders (External Systems & Services)

- **Map Provider:** A critical external actor (simulated in this backend via a DummyMapProvider) responsible for validating addresses, calculating straight-line spherical distances using the Haversine formula, and providing reliable ETA metrics based on average city speeds.
- **Payment Gateway:** A financial third-party system responsible for validating and executing passenger transactions. It verifies payment methods (e.g., credit card approvals), safely deducts the platform's 20% commission fee, and securely deposits the remaining earnings into the driver's account.

2.2 Functional Requirements

A. User Registration and Authentication

To ensure security and proper role allocation, the system must allow users to register an account by specifying their role as either a "Passenger" or a "Driver". During registration, the system validates the uniqueness of the username, securely hashes the password using BCrypt, and stores the credentials in the database. Once registered, users must log in with their credentials. If the provided username does not exist or the password is incorrect, the system will deny access.

B. Driver Availability and Vehicle Management

The system must provide drivers with the ability to register specific vehicles (by entering the license plate and vehicle model) and add them to their personal garage. Before they can accept any trip assignments, drivers must select an active vehicle from their garage, input their current location, and toggle their status to

"online". The system will then add the driver to the available dispatcher list.

C. Ride Requesting and Dynamic Dispatching

Passengers can initiate a ride by inputting their origin, destination, and acceptable vehicle types (e.g., standard car or SUV). The system interacts with a Map Provider to validate the addresses, calculate the straight-line distance, and estimate the time of arrival (ETA). Subsequently, the system calculates an estimated fare and broadcasts the ride request via push notifications to all eligible and available drivers located within a 5 km radius.

D. Ride Execution and State Tracking

The system must manage and track the lifecycle of every ride using a strict state machine. When a driver accepts a broadcast request, the system prevents other drivers from taking the order and changes the ride state to "Accepted". The driver can then click "Start ride" to shift the status to "In Transit". Upon reaching the destination, the system requires confirmation of arrival (supporting both single and dual confirmation modes) to officially mark the ride as "Completed".

E. Payment Processing and Trip History

After a ride is confirmed as completed, the system evaluates the trip to ensure the actual route did not severely deviate from the initial estimate. The system's payment gateway then calculates the final total fare, deducts the platform's 20% commission fee, and credits the remaining 80% to the driver's account. Finally, the system generates a permanent ride record—including the ride ID, usernames, fare, and completion time—allowing both the passenger and the driver to view their past trip histories at any time.

2.3 Non-functional Requirements

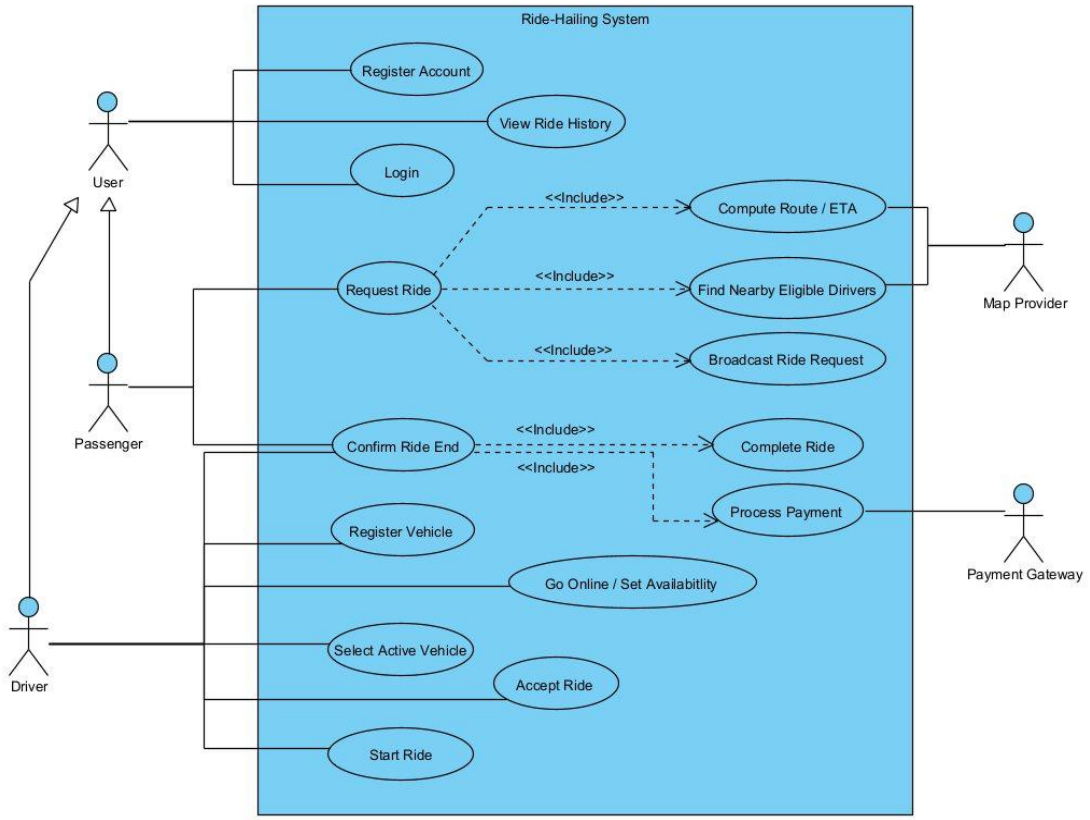
- **Security & Privacy**
  - **User Authentication:** To guarantee secure access and protect user identities, all passengers and drivers are required to authenticate themselves by logging into the system using a registered username and a

secure password.

- **Data Privacy and Authorization:** The system must strictly enforce data isolation. Sensitive information, such as trip histories, route details, and payment transactions, must remain strictly confidential and visible only to the specific passenger and driver involved in that ride. The system must actively prevent unauthorized access, ensuring users cannot view or manipulate the records of others.

- **Usability (User-Friendliness)**
  - **Intuitive Interface Design:** To deliver a seamless user experience and minimize the learning curve, the system's interface must be intuitive. All navigational elements and action buttons must be clearly labeled with concise, descriptive wording so that users can instantly understand their functions without requiring a manual.
  - **Information Accessibility:** The system must present ride histories and payment records in a highly organized, detailed, and easily readable format. This ensures that users can effortlessly navigate, retrieve, and audit their past platform activities at any given time.

2.4 Use Case Diagram



The above diagram shows the Use Case Diagram of the Ride Hailing System. There are 4 actors: Passenger, Driver, Map Provider, Payment gateway. Passenger and Driver is the primary actor; User is the generalization of Passenger and Driver.

2.5 Use Case Specification

Use Case Name	**Register account**	
Actor	User (Passenger, Driver)	
Description	The use case describes the process of a user registering an account. After completion, the user is told that the account is successfully registered.	
Typical course of events	Actor Action	System Response
	Step 1: User enters the role, username, and password.	

		Step 2: System confirms that the username is unique. Step 3: System hashes password. Step 4: System creates a user account and puts user account information in the database. Step 5: System shows successful registration.
Alternative course of events	Step 2a: If the username is not unique, registration fails. Use case ends. Step 4a: If the role is not "passenger" or "driver", user account information is not put in the database. Registration fails and use case ends.	
Post-condition	The account is successfully registered.	

Use Case Name	**Login**	
Actor	User (Passenger, Driver)	
Description	The use case describes the process of a user logging into the system. After completion, the user is told that he/she can successfully login to the system.	
Typical course of events	Actor Action	System Response
	Step 1: User inputs username and password, and clicks "Log in".	Step 2: System checks whether the username exists, and the password is correct. Step 3: System shows successful login.
Alternative course of events	Step 2a: If the username does not exist, login fails. Use case ends. Step 2b: If the password is incorrect, login fails. Use case	

	ends.	
Pre-condition	The account has been registered already.	
Post-condition	User can successfully login to the system.	

Use Case Name	**View ride history**	
Actor	User (Passenger, Driver)	
Description	The use case describes the process of a user asking for viewing ride history content. After completion, the user is shown all of the past ride information.	
Typical course of events	Actor Action	System Response
	Step 1: User asks to view ride history.	Step 2: System lists all ride history content, including ride ID, passenger username, driver username, fare, and end time of the ride.
Pre-condition	Driver has logged in.	
Post-condition	The system retrieves and displays all ride records associated with the driver.	

Use Case Name	**Register vehicle**	
Actor	Driver	
Description	The use case describes the process of a driver registering a vehicle and adding a vehicle into their own garage. Later, the user is informed of the process's completion.	
Typical course of events	Actor Action	System Response
	Step 1: Driver registers a vehicle by providing license plate and model.	Step 2: System adds the registered vehicle to the vehicle list.

		Step 3: System shows that the vehicle is added to the driver's garage.
Pre-condition	Driver has logged in.	
Post-condition	A vehicle is registered and added to the driver's garage.	

Use Case Name	**Request ride, Compute route / ETA, Find nearby eligible drivers, Broadcast ride request**	
Actor	Passenger, Map Provider	
Description	The use case describes the process of a passenger requesting a ride.	
Typical course of events	Actor Action	System Response
	Step 1: Passenger provides requirements, which include origin, destination, acceptable vehicle types, and pricing strategy (Standard pricing). Passenger clicks "Confirm standard".	
		Step 2: System confirms that passenger has no unpaid balance and has valid payment method. Step 3: Invoke "Compute route / ETA" use case.
	Step 3a: Map provider checks whether the address of origin and destination are valid. Step 3b: Map provider calculates straight line distance between origin and destination, and driver's estimated time of arrival. A route is created.	

	Step 6b: Map provider calculates straight line distance between driver's current location and pickup location.	Step 4: System creates a new ride, which includes passenger requirements, route, ride ID, and a requested state. Step 5: System calculates estimated fare. Step 6: Invoke "Find nearby eligible drivers". Step 6a: System checks whether drivers are available, and the driver's vehicle type matches passenger requirements. Step 6c: System adds the driver to the eligible driver list if the distance is within 5 km. Step 7: Invoke "Broadcast ride request": System broadcasts the details of the ride to all eligible drivers.
Alternative course of events	Step 2a: If passenger has unpaid balance or no valid payment method, he/she will be told to check payment method and unpaid balance. Passenger cannot get a ride and use case ends. Step 3ai: If the address of origin or destination is invalid, no route is provided. Passenger cannot get a ride and use case ends. Step 7a: If there are no eligible drivers after 3 minutes, the passenger finishes step 1 with pricing strategy changed to "Surge pricing". Then, system goes through step 2 to 7 again.	
Pre-condition	Passenger has logged in.	
Post-condition	Passenger gets a ride.	

Use Case Name	**Go online / Set availability, Select active vehicle**	
Actor	Driver	
Description	The use case describes the process of a driver becoming eligible to accept a ride.	
Typical course of events	Actor Action	System Response
	Step 1: Driver inputs license plate of the selected vehicle and current location.	Step 2: System checks whether the selected vehicle is in the driver's garage. Step 3: System sets driver's active vehicle, makes driver available to accept ride, updates driver's current location, and adds the driver to the online driver list. Step 4: System shows that the driver is online.
Alternative course of events	Step 2a: If the selected vehicle is not in the driver's garage, driver remains unavailable to accept ride. Use case ends.	
Pre-condition	Driver has logged in and has a vehicle in garage.	
Post-condition	Driver sets the availability to true, selects a vehicle, and becomes eligible to accept a ride.	
Use Case Name	**Accept ride**	
Actor	Driver	
Description	The use case describes the process of a driver getting an order requested by a passenger.	
Typical course of events	Actor Action	System Response
		Step 1: System shows a push notification about the pickup location, ride distance, and estimated fare.

	Step 2: Driver clicks "accept".	Step 3: System checks if the ride has no driver. Step 4: System assigns the ride to driver, changes the state of the ride from "requested" to "accepted", and outputs that the ride is assigned to the driver. Step 5: System sets the driver as unavailable to accept order and starts navigation.
Alternative course of events	Step 2a: If driver clicks "reject", the ride is not accepted and use case ends. Step 3a: If the ride already has a driver, system informs the driver that the order has already been taken by another driver. Use case ends.	
Pre-condition	Driver has logged in and is available to accept a ride.	
Post-condition	Driver gets the order and drives to the pickup location.	

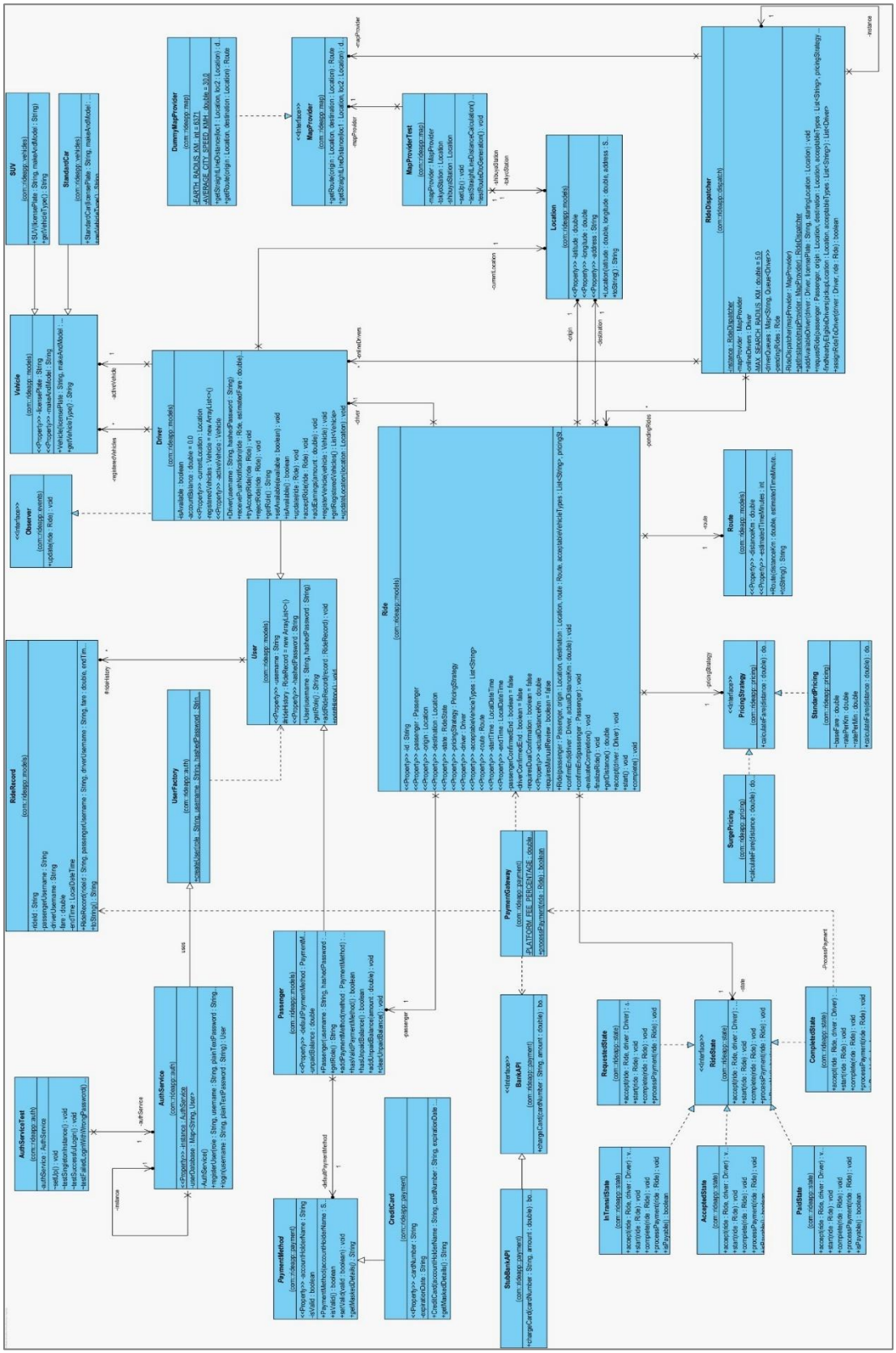
Use Case Name	**Start ride**	
Actor	Driver	
Description	The use case describes the process of a driver starting a ride.	
Typical course of events	Actor Action	System Response
	Step 1: Driver clicks "start ride".	Step 2: System sets current time as the start time of the ride. Step 3: System changes the state of the ride from "accepted" to "in transit".
Pre-condition	Driver has logged in, accepted a ride, and arrived at the pickup location.	
Post-condition	Driver starts the ride and is bringing the passenger to the destination.	

Use Case Name	**Confirm ride end, Complete ride, Process Payment**	
Actor	Passenger, Driver, Payment Gateway	
Description	The use case describes the process of passenger and driver confirming the end of ride, and payment gateway dealing with the payment process.	
Typical course of events	Actor Action	System Response
	Step 1: Driver and passenger click "Confirm Arrival".	Step 2: In single confirmation mode, the system checks whether driver confirms ride end. Step 3: System checks whether the route severely deviates by comparing estimated distance and final distance. Step 4: Invoke "Complete ride" use case: System sets current time as end time and changes the state of the ride from "in transit" to "completed". Step 5: Invoke "Process payment" use case. Step 5a: System confirms that the ride is payable. Step 5b: System calculates the total fare. Step 5c: System gets passenger's default payment method. Step 5d: System checks whether the passenger pays by credit card. Step 5f: Payment Gateway checks whether the bank approves the credit card. Step 5g: System calculates

	Step 5e: Payment gateway approves the credit card.	platform fee and driver earnings, and adds driver earnings to driver's account. Step 5h: System creates a new ride record, which includes a ride ID, passenger username, driver username, fare, and end time of the ride. System adds the ride record to passenger account and driver account. Step 5i: System changes the state of the ride from "completed" to "paid". Step 6: System changes the driver's availability to "avaliabe", so driver can accept another order later.
Alternative course of events	Step 2a: If driver has not confirmed end, system waits until driver confirms end to continue. Step 2b: If dual confirmation mode is used, the system checks whether both driver and passenger confirm ride end. If either one of them has not confirmed end, system waits until both confirms end to continue. Step 3a: If the route severely deviates, there will be manual review by customer service. There is no need to process payment until the problem is solved. Step 5ai: If the ride is not payable, use case ends. Step 5di. If the passenger does not pay by credit card, the system outputs that the payment method is unsupported. Step 5ei: If the credit card is declined due to insufficient funds, the bank's approved state remains false. Step 5fi. If the bank does not approve the credit card, the system adds the total fare of the current ride to passenger's unpaid balance. Process payment fails and use case ends.	
Pre-condition	Passenger and driver have logged in. Passenger arrives at the destination.	
Post-condition	Passenger arrives at the destination and pays. Driver earns money. System extracts platform fee. The ride ends.	

3. System Design

3.1 Class Diagram



3.2 Description of Class Diagram

This class diagram shows the architecture of our ride-hailing system.

Central of the system is the Ride class, which contains key elements such as Passenger, Driver, RideState, Route, Location, and PricingStrategy. Ride shows the full lifecycle of a trip, including start time, end time, estimated fare, and payment processing.

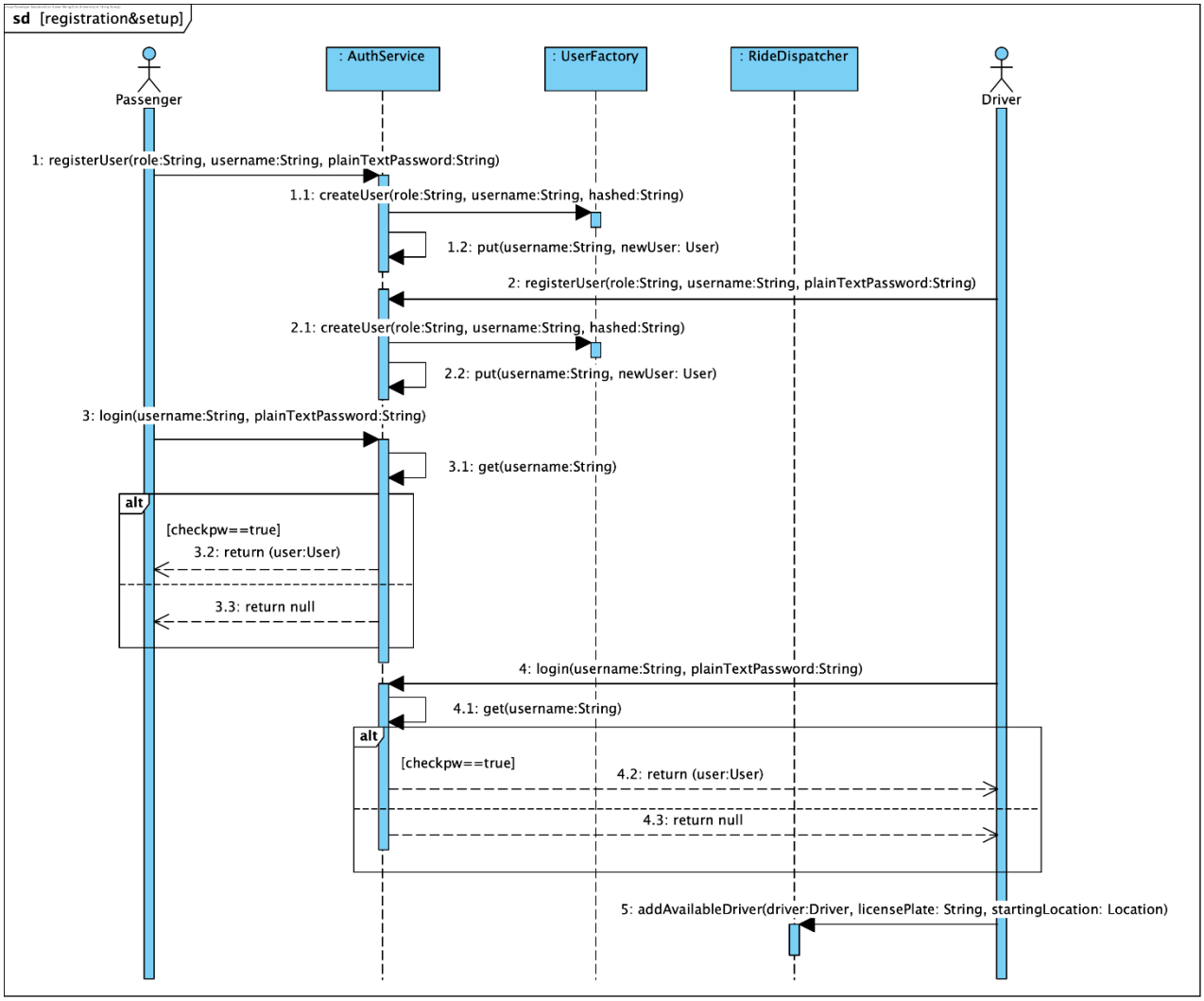
User management is handled by User, extending to Passenger and Driver. Passengers have their own payment methods. Drivers have their own availability, current location, registered vehicles, and active vehicle status. AuthService supports user registration and login, assisted by User Factory.

The ride lifecycle state is modeled using RideState interface and the five states: RequestState, AcceptedState, InTransitState, CompletedState, and PaidState. These states show accepting, starting, completing, and paying for rides.

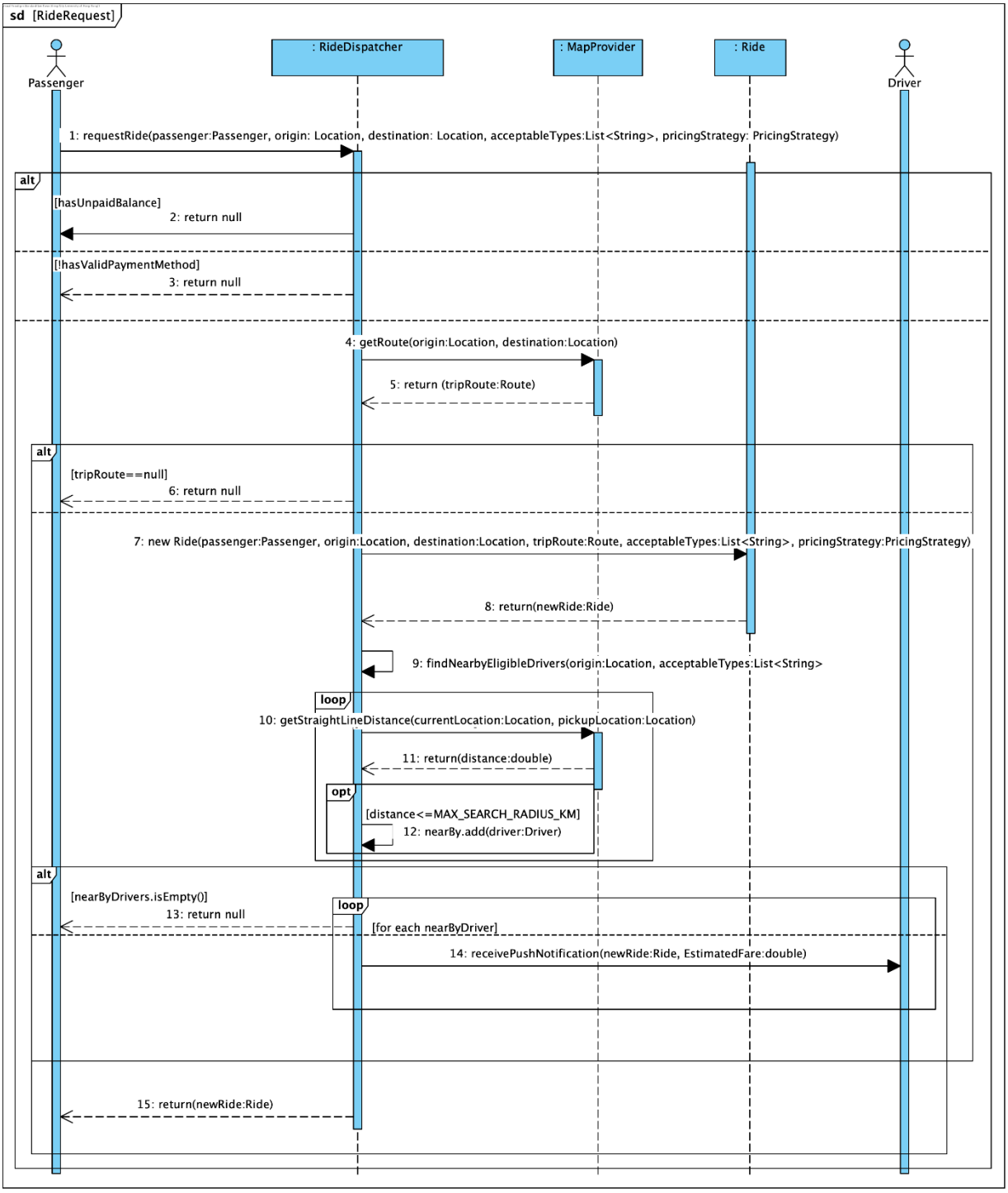
There are also some supporting components: RideDispatcher for matching drivers to rides, MapProvider for route calculation and distance, and some pricing strategies. RideRecord for store trip history for both Passenger and Driver.

3.3 Sequence Diagrams / Object Interactions

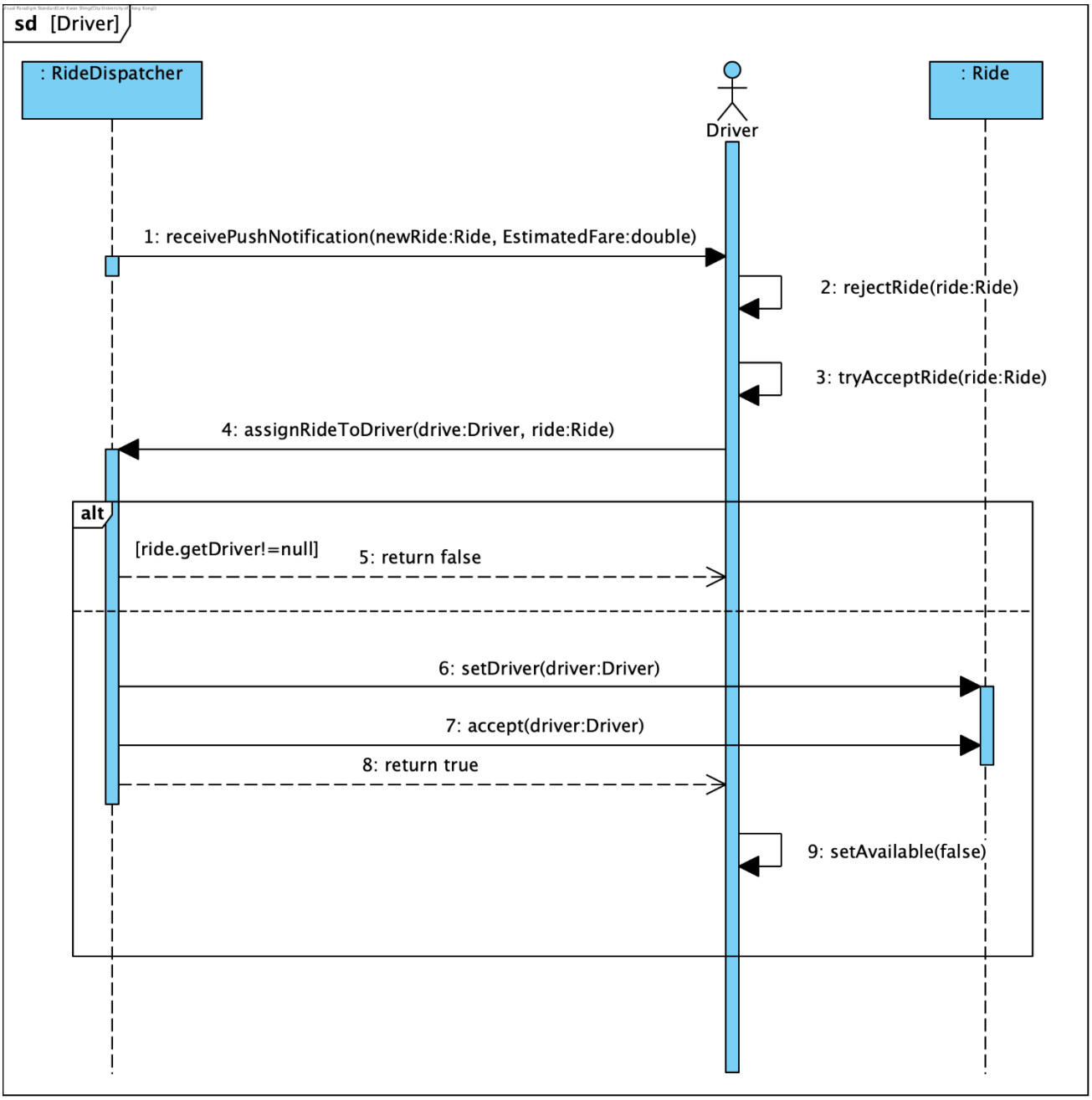
3.3.1 Registration & Setup



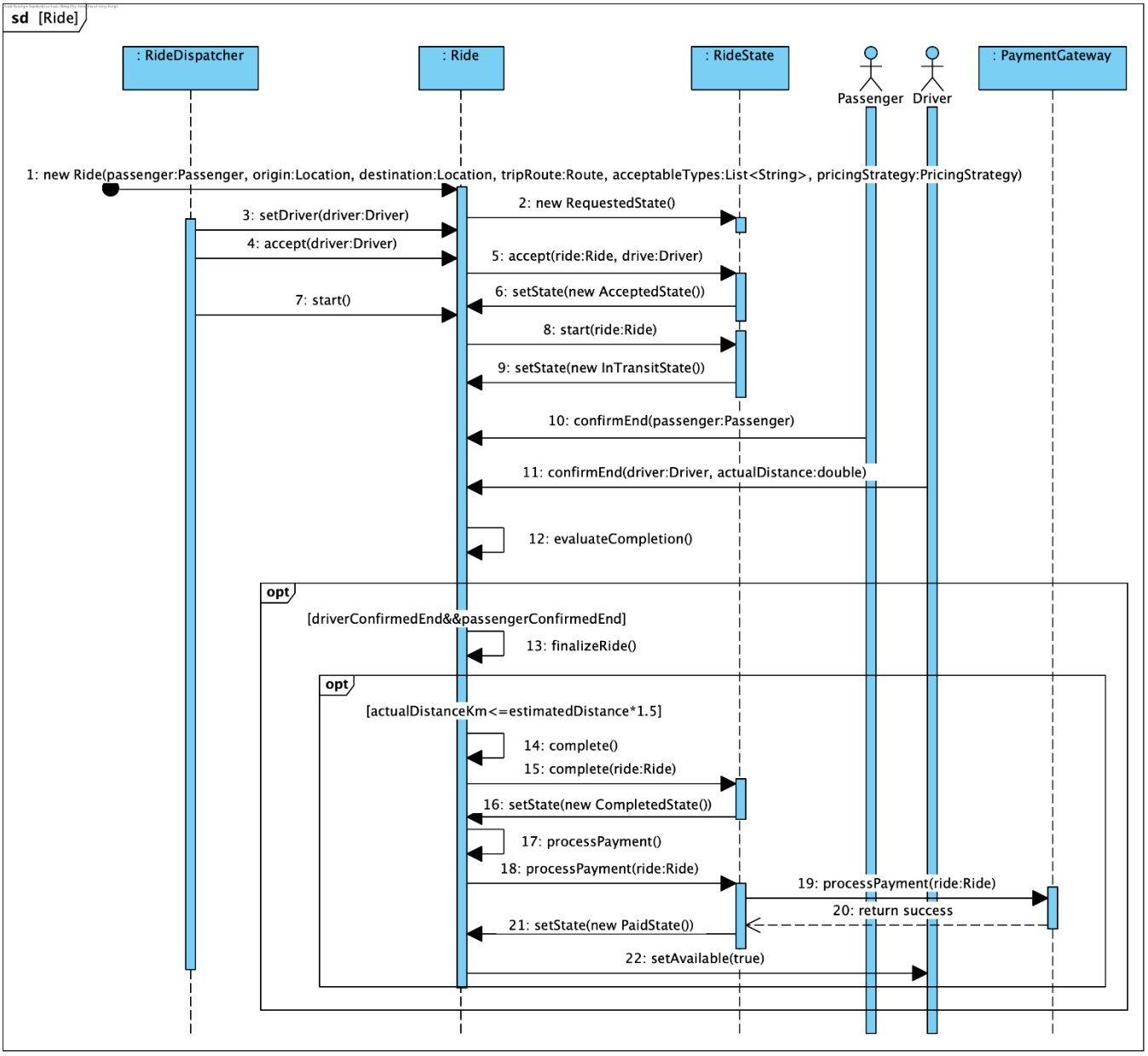
3.3.2 Ride Request



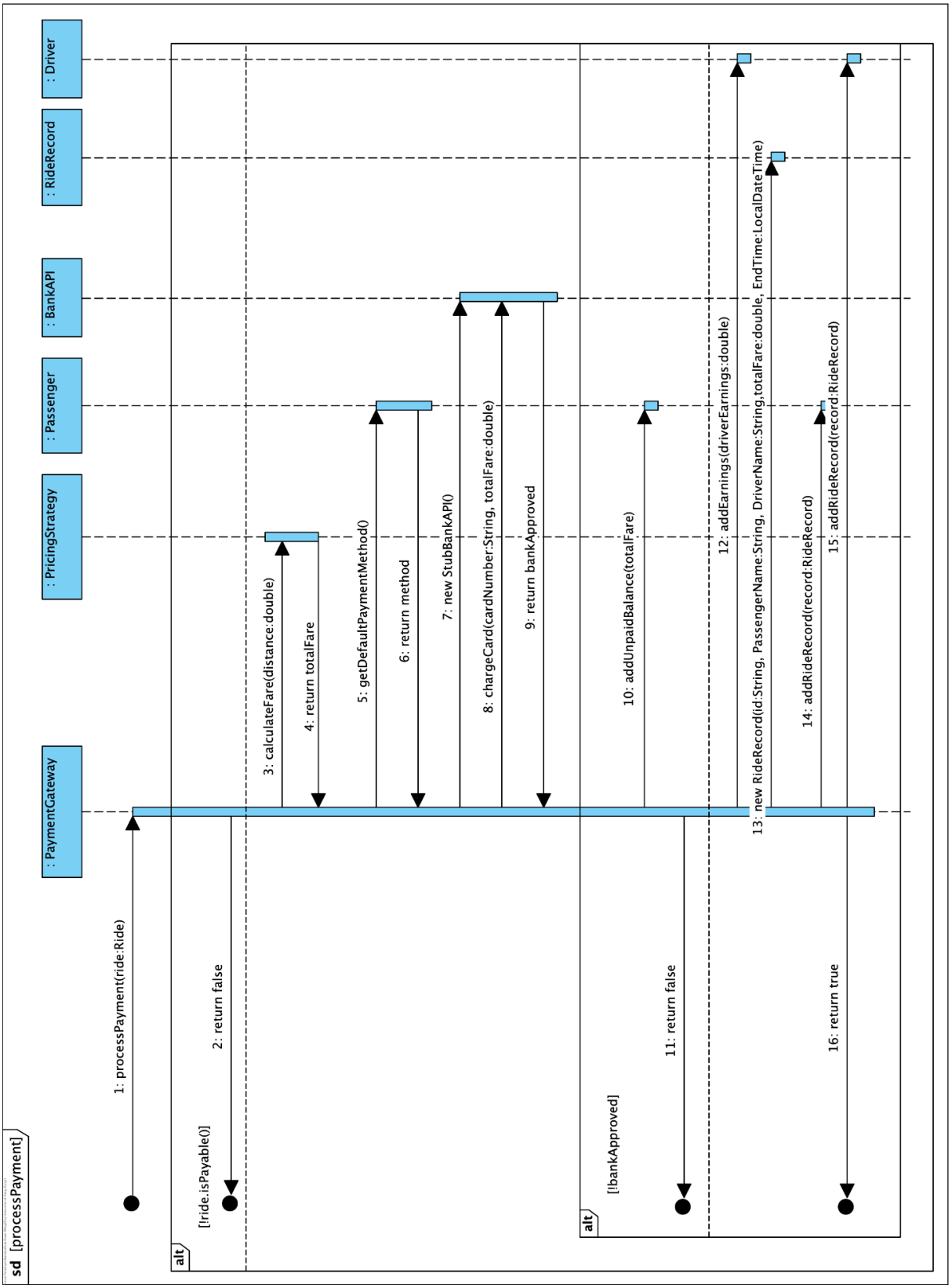
3.3.3 Driver View



3.3.4 Ride


sd [Ride]
: RideDispatcher
: Ride
: RideState
Passenger
Driver
: PaymentGateway
1: new Ride(passenger:Passenger, origin:Location, destination:Location, tripRoute:Route, acceptableTypes:List<String>, pricingStrategy:PricingStrategy)
2: new RequestedState()
3: setDriver(driver:Driver)
4: accept(driver:Driver)
5: accept(ride:Ride, drive:Driver)
6: setState(new AcceptedState())
7: start()
8: start(ride:Ride)
9: setState(new InTransitState())
10: confirmEnd(passenger:Passenger)
11: confirmEnd(driver:Driver, actualDistance:double)
12: evaluateCompletion()
opt
[driverConfirmedEnd&&passengerConfirmedEnd]
13: finalizeRide()
opt
[actualDistanceKm<=estimatedDistance*1.5]
14: complete()
15: complete(ride:Ride)
16: setState(new CompletedState())
17: processPayment()
18: processPayment(ride:Ride)
19: processPayment(ride:Ride)
20: return success
21: setState(new PaidState())
22: setAvailable(true)

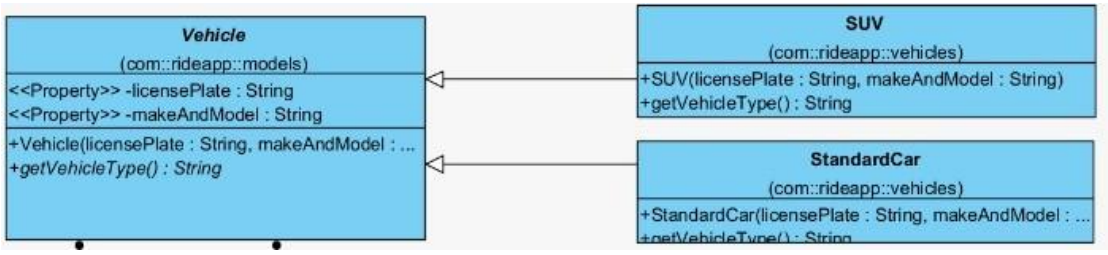
3.3.5 Process Payment



4. Utilization of Software Design Principles and Patterns

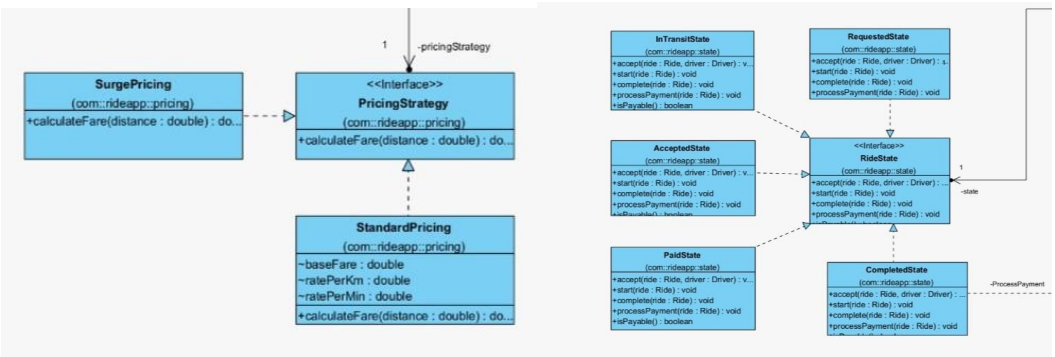
4.1 Applied Design Principles

4.1.1 Open-Closed Principle / Liskov Substitution Principle



One application is seen in the *Vehicle* class, which represents the documentation of a driver's vehicle in real life. All vehicles share the same properties, such as having a license plate, a make a model designation, and as implied in the system, able to transport a passenger. In our prototype, we have subclasses *StandardCar* and *SUV*, which current only serve as supplementary information for passenger preference. It is possible in a future implementation that more car types can be introduced, and more properties can be integrated into the subclasses, such as cargo carrying capacity. However, these potential future extensions do not violate upon the core properties of a car needed by the system, which having is a retrievable license plate number and a referenceable vehicle type.

4.1.2 Dependency Inversion Principle



We enforce the dependency inversion principle in our state and strategy patterns.

Rather than trying to reference a specific instance of state, we allow the state/strategy to change and modify the properties of the *Ride* object it belongs in to perform its functions. Similarly, the *Ride* does not check for a specific pricing strategy to calculate fees, but simply calls upon *PricingStrategy* to calculate fees, and allows the specific subclass currently used to provide the result based on its own algorithm.

4.1.3 Single Responsibility Principle

Most entities in the system handle one single responsibility. For example, user login is done by the AuthService, while user creation is separated into the UserFactory. User account manager is handled by the AuthService, while the actual pairing is done by the RideState. Within rides, certain jobs are also separated, where properties of the ride are separated into ridestates, and pricing into pricingstrategies.

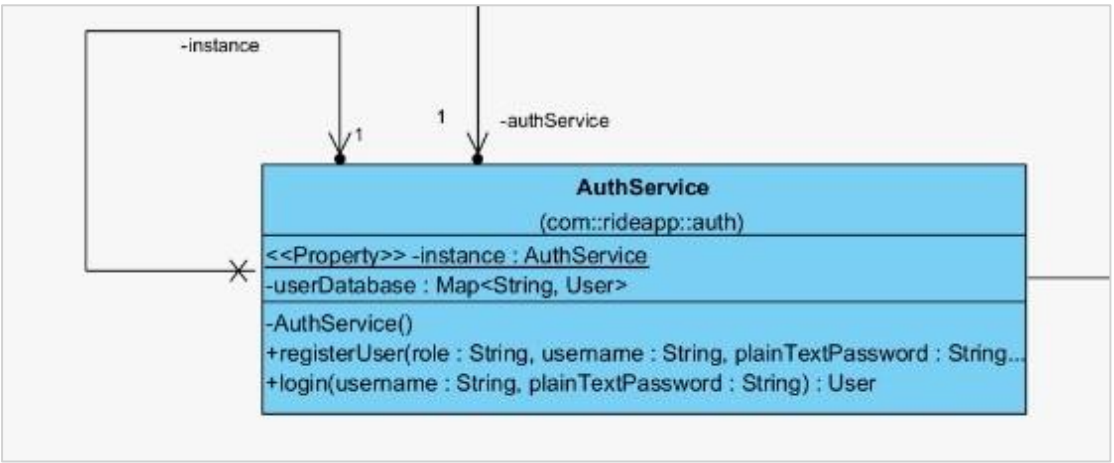
Some components of the system technically violate the single responsibility principle, such as the RideStates, which handles the ride's discoverability to other drivers, the availability of payment, and keeping track of whether the ride is considered complete. However, since each state in the program refers to an actual state of the actual ride, and these properties

4.1.4 Law of Demeter

Most entities in the system have very limited knowledge of others. For example, all passengers are not aware of the existence of any driver, unless it has created a ride object which is accepted by a driver, and so only that driver is known to the user. No user can be aware of the AuthService beyond the login step, and thus cannot iterate through its entire database to look for a driver. It is technically not even aware of the existence of the RideDispatcher's access to driver records, only that it has access to the given Ride record. Conversely, drivers do not know the full list of passengers, just those provided by the RideDispatcher. The RideDispatcher handles the matchmaking, taking and providing only information needed by the driver or the passenger to create a match.

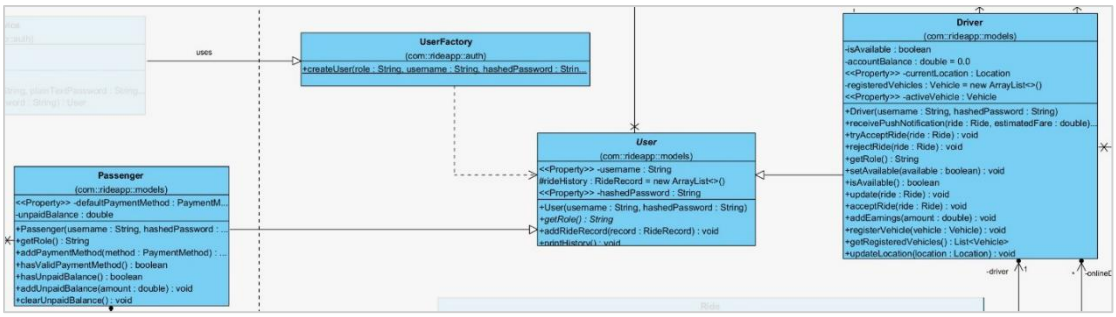
4.2 Applied Design Patterns

4.2.1 Singleton Pattern



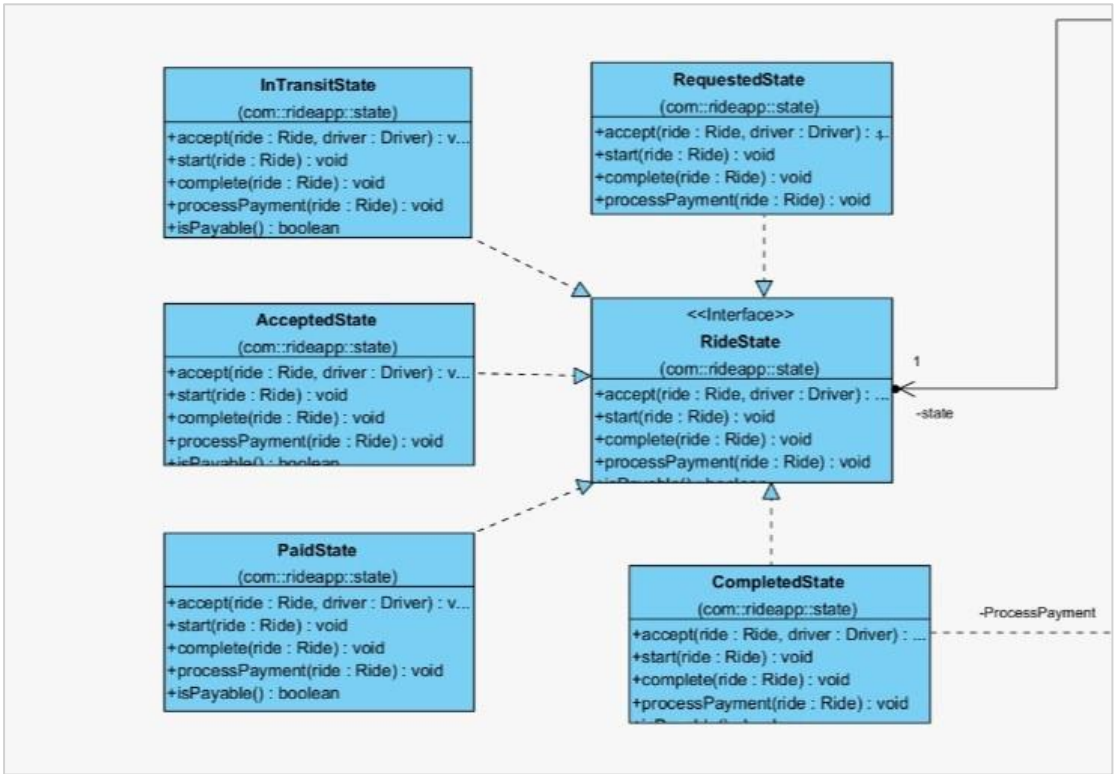
The *AuthService* system is a singleton that handles user login data and authenticates all user logins with data in its own parameters. The *RideDispatcher* system records all available drivers and passengers, keeps them within one single instance, then retrieves the records whenever a passenger requests a ride to match a driver that matches requirements.

4.2.2 Factory Pattern



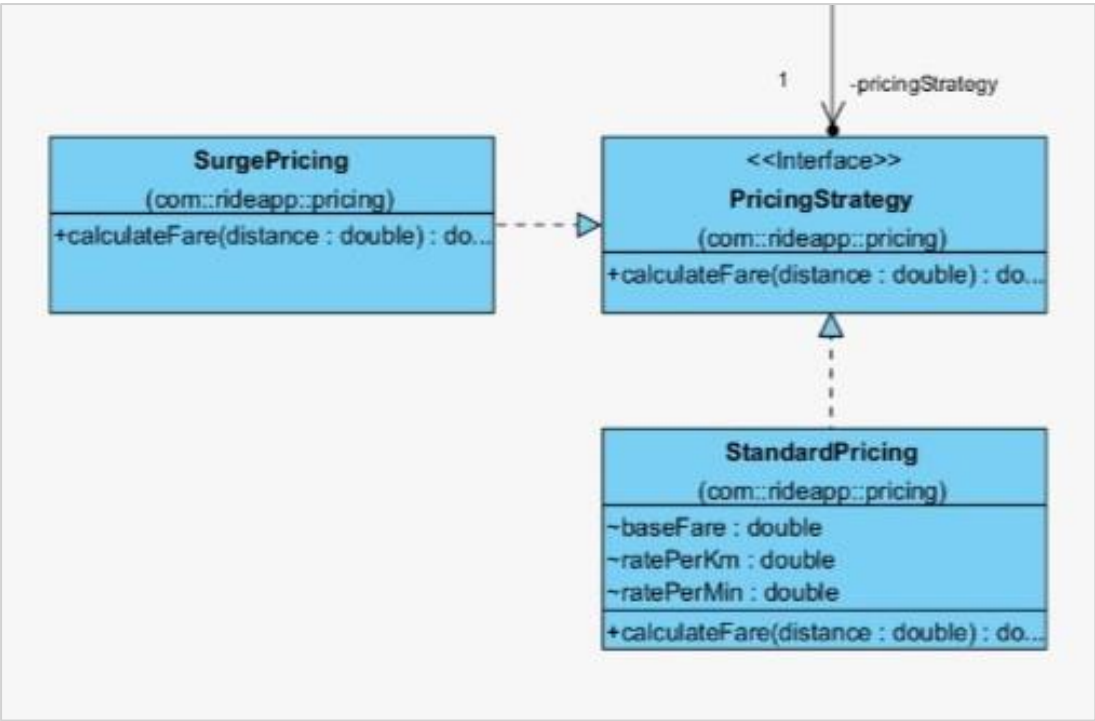
The factory pattern is implemented in the *UserFactory*, where when new users register, we determine based on input whether the new *User* instance should be created as a *Driver* or a *Passenger*.

4.2.3 State Pattern



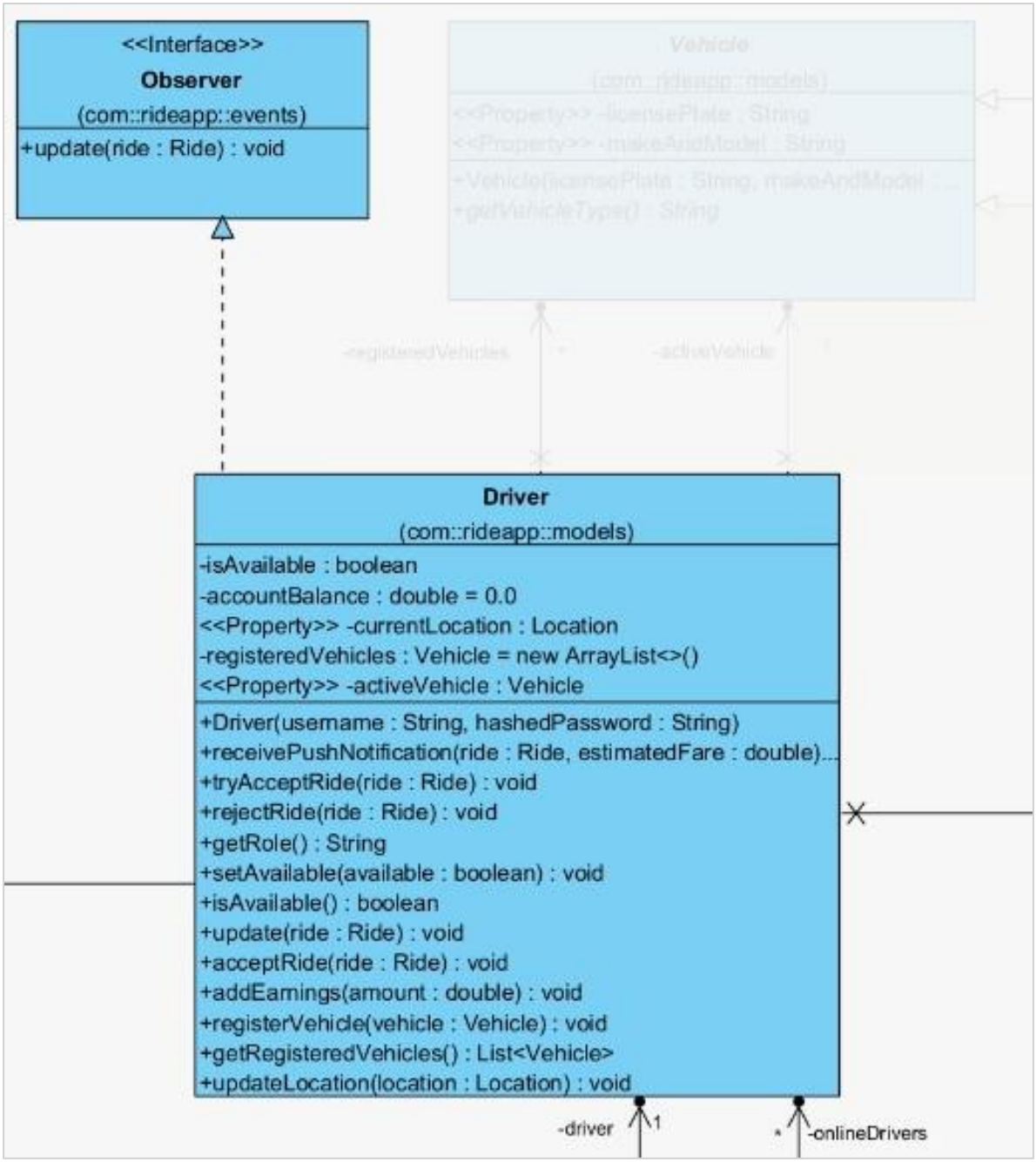
The state pattern is applied in the *RideState* component, which represents the state the Ride object is in, which dictates what interactions the object can perform. For example, a “requested” state would allow the ride to be found by drivers through the dispatcher system, while a “completed” state would allow the ride to accept a payment.

4.2.4 Strategy Pattern



Here, PricingStrategy determines how the fare of the ride is determined, by passing parameters to the calculation function and changing the price calculation method to be used. Under certain circumstances, surge pricing may be adopted, which will lead to the need to calculate a higher charge rate.

4.2.5 Observer Pattern



The observer pattern is used to inform drivers about available rides. The RideDispatcher constantly updates driver, passenger and ride data, which then notifies drivers in relevant areas about updates. Drivers that are actively seeking passengers to pick up can then request a copy of the latest data to choose passengers to pick up.

4.3 Test code

```java
public class PaymentTest {


    @Test
    void testSuccessfulPayment() {
        assertTrue(PaymentGateway.processPayment(RideStub),message: "This ride should be valid");
    }

    @Test
    void testFailedNotPayable() {
        assertFalse(PaymentGateway.processPayment(RideStubUnpayable),message: "Payment should only be calculated for
    }
    @Test
    void testFailedNotCard() {
        assertFalse(PaymentGateway.processPayment(RideStubNotCredit),message: "Payment should only accept credit car
    }

    @Test
    void testFailedInSuffBal() {
        assertFalse(PaymentGateway.processPayment(RideStubNoMoney),message: "Payment should be denied if insufficier
    }
```

```java
class AuthServiceTest {
    @BeforeEach
    void setUp() {
        authService = AuthService.getInstance();
        // Since it's a Singleton, it persists across tests.
        // In a real app, we'd add a method to clear the map for test isolation.
        authService.registerUser(role: "PASSENGER", username: "unit_tester", plainTextPassword: "Secret12
    }

    @Test
    void testSingletonInstance() {
        AuthService instance1 = AuthService.getInstance();
        AuthService instance2 = AuthService.getInstance();

        assertSame(instance1, instance2, message: "AuthService must return the exact same instance in me
    }

    @Test
    void testSuccessfulLogin() {
        User loggedIn = authService.login(username: "unit_tester", plainTextPassword: "Secret123!");

        assertNotNull(loggedIn, message: "Login should succeed with correct credentials");
        assertEquals(expected: "unit_tester", loggedIn.getUsername());
        assertEquals(expected: "PASSENGER", loggedIn.getRole());
    }

    @Test
    void testFailedLoginWithWrongPassword() {
        User loggedIn = authService.login(username: "unit_tester", plainTextPassword: "WrongPassword");

        assertNull(loggedIn, message: "Login should fail and return null with incorrect credentials");
    }

    @Test
    void testFailedLoginWithUnknownUser() {
        User loggedIn = authService.login(username: "ghost_user", plainTextPassword: "Secret123!");
```

```java
import com.rideapp.models.Location;
import com.rideapp.models.Route;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class MapProviderTest {
    private MapProvider mapProvider;
    private Location tokyoStation;
    private Location shibuyaStation;

    @BeforeEach
    void setUp() {
        mapProvider = new DummyMapProvider();
        tokyoStation = new Location(latitude: 35.6812, longitude: 139.7671, address: "Tokyo Station");
        shibuyaStation = new Location(latitude: 35.6580, longitude: 139.7016, address: "Shibuya Station")
    }

    @Test
    void testStraightLineDistanceCalculation() {
        double distance = mapProvider.getStraightLineDistance(tokyoStation, shibuyaStation);

        // The real-world straight-line distance is roughly 6.5km.
        // We use a delta of 0.5km to account for slight Haversine rounding differences.
        assertEquals(expected: 6.5, distance, delta: 0.5, message: "Distance should be approximately 6.5k
    }

    @Test
    void testRouteDtoGeneration() {
        Route route = mapProvider.getRoute(tokyoStation, shibuyaStation);

        assertNotNull(route, message: "Route DTO should not be null");
        assertTrue(route.getDistanceKm() > 0, message: "Distance should be positive");
        assertTrue(route.getEstimatedTimeMinutes() > 0, message: "ETA should be positive");
    }
}
```

```java
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class RideDispatcherTest {
    private RideDispatcher dispatcherInstance;

    @BeforeEach
    void setUp() {
        dispatcherInstance = RideDispatcher.getInstance(mapProvider: null);
    }

     @Test
    void testSuccessReq(){
        dispatcherInstance.addAvailableDriver(drive, licensePlate: "D123456", origin1);
        assertNull(dispatcherInstance.requestRide(passengerStub, origin1, dest1, TypeStub, new StandardPricing()),me
    }
    @Test
    void testPayment(){
        assertNull(dispatcherInstance.requestRide(passengerInv, origin1, dest1, TypeStub, new StandardPricing()),me
    }
    @Test
    void testNoMoney(){
        assertNull(dispatcherInstance.requestRide(passengerbroke, origin1, dest1, TypeStub, new StandardPricing()),me
    }
    @Test
    void testNoDriversAvail(){
        assertNull(dispatcherInstance.requestRide(passengerStub, origin1, dest1, TypeStub, new StandardPricing()),mes
    }

    @Test
    void testRideTaken() {
        assertFalse(dispatcherInstance.assignRideToDriver(drive, ridestubtaken),message: "Driver should not be able to
```

Attached are some snippets of testcases. Most modules in our system have been tested to ensure proper responses to situations, including rejections to expected failures.

The first image shows test for the payment portal, which should accept a payment if the bank returns a success, or reject a payment under certain circumstances, such as the number not being valid, the card being declined by the bank, or the ride in question should not be payable.

The second image shows the authentication service, which allows a user to login with the correct credentials, with corresponding rejections for incorrect password, non-existent username. It also ensures that it consistently returns the correct instance of user for each login with the same credentials.

The third image shows the map route calculation system, which should calculate the distance between the origin and destination, and estimates the time before arrival. It asserts that the calculation algorithm provides the correct output for a few test locations, and does not return invalid data such as null values or negative values.

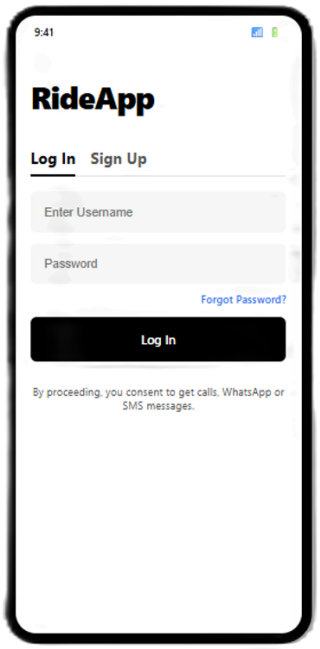
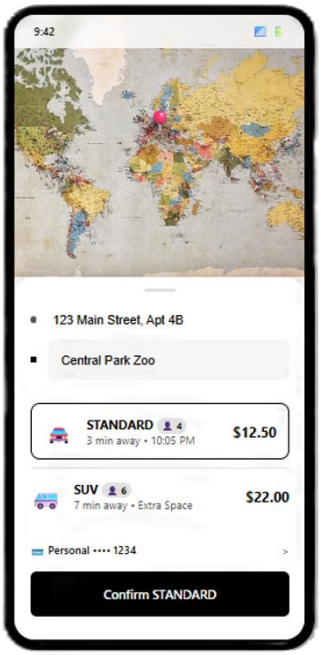
The fourth image shows some tests for the ride dispatcher, where the ride dispatcher takes in stubs of rides, which simulate different ride states and users.

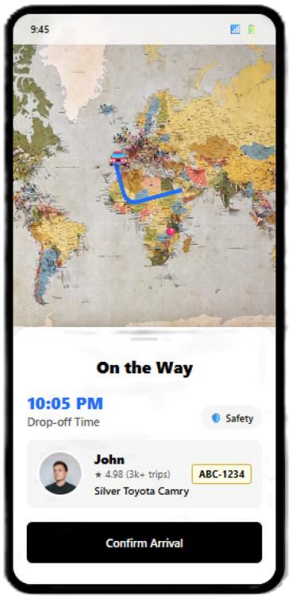
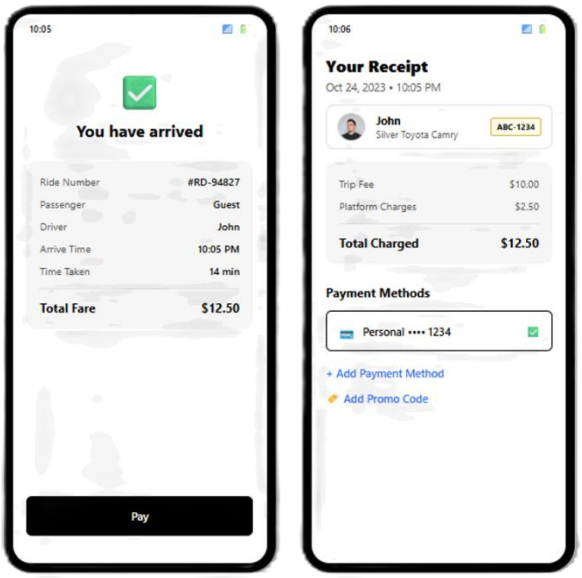
For example, the dispatcher should not accept rides from users with outstanding payments, or users with no valid payment method, or match them with drivers too far away, or those that do not fit certain demands. It should also not update drivers with null/nonexistent rides or accept rides with missing information.

5. System Prototype

5.1 UI Design & User Flow

Our team has developed a frontend prototype that enables users to interact with the ride-hailing system. The following table presents the interface design and corresponding user flow.

	
1. Users login to the app by entering username and password. Then, click “Log in”.	2. The passenger selects the pickup location and destination by typing or tapping the places on the map. 3. The passenger chooses the acceptable vehicle types and click “confirm”. 4. Later, a driver will accept the order and drive to the pickup location.

	
5. After picking up the passenger, when the driver is driving to the destination, the passenger can see the estimated drop-off time, driver information and estimated route on the map. 6. After arriving at the destination, the driver clicks “confirm arrival”.	7. The passenger can check details of the ride, such as arrival time, time taken by the ride, and total fare. 8. After confirming the details are correct, the passenger clicks “pay” and chooses a credit card to pay.

6. Project Management

6.1 Lifecycle Management & Development Process

For schedule and task management, the project adopts an iterative development approach integrated with Continuous Integration (CI) practices. By utilizing GitHub for centralized code management and version control, the team ensures that all software components are continuously integrated and synchronized. Rather than following a strict linear process, the project incorporates continuous testing and feedback loops; code is routinely reviewed and immediately refined based on test case results (as executed in our development phases). This iterative, CI-driven model allows the team to frequently verify that the current prototype

matches user requirements, mitigate risks early, and ensure a stable codebase before advancing to subsequent phases like system diagramming and UI generation.

6.1 Team Roles & Work Allocation

Members	**roles**	**Work to do**
Chan Kin Ching	Project Manager	Planning
Chan Shing Shun	Assistant Project Manage	assist PM
Lee Kwan Shing	Assistant Project Manage	assist PM
Fan Wai Tsun	Software Designers	Produce document
Heung Chi Ho	Software Designers	Produce document
Pan Qi Yuan	Software Designers	Design and test

6.2 Weekly Log

The weekly log reflects iterative refinement of system design, especially in state modeling and dispatch logic.

Week	Weekly Activity Log	Completed By
Week 1	Recruit Project Memebers to Form a Team.	All team members
Week 2	Assign roles and discuss project topics; conduct a feasibility analysis; define the scope and collaboration model	All team members
Week 3	Set up shared documents to shared informations, and select GitHub for code management and hosting.	All team members
Week 4	Set internal timelines and deadlines	All team members
Week 5	Drafting proposals and discussing use case descriptions, as well as discussing the implementation details of modules and functions and the design patterns used	Pan Qi Yuan，Chan Kin Ching,Chan Shing Shun
Week 6	In-person meeting: Finalize the proposal and assign tasks for code development based on the proposal	All team members
Week 7	Review the code internally and create test cases to run tests,	Heung Chi Ho,Lee Kwan Shing,Pan Qi Yuan
Week 8	Continuously improve the code based on test case results	Heung Chi Ho,Lee Kwan Shing
Week 9	Generate user interfaces, class diagrams, and sequence diagrams simultaneously	All team members
Week 10	zoom meeting: Review diagrams and reports internally to ensure consistency and completeness, then create presentation user interfaces, class diagrams, and sequence diagrams.	All team members
Week 11	Write reports and continue working on slides	All team members
Week 12	Complete the report and presentation	All team members
Week 13	Review report and practice presentation before delivering	All team members

6.3 Risk Management

To ensure the successful delivery of the Ride-hailing System prototype within the strict 3 months development timeline, the team adopted an iterative risk assessment strategy integrated into our Continuous Integration (CI) and testing pipelines. The key identified risks and their corresponding mitigation strategies are outlined below:

1. Technical Feasibility and External Dependency Constraints

- **Risk:** Because the project scope excludes integration with live, rate-limited mapping APIs (e.g., live Google Maps), there is a risk of route calculation inaccuracies that could disrupt the dispatching logic and ETA generation.
- **Mitigation:** The team developed a DummyMapProvider that mathematically simulates routing by applying the Haversine formula to calculate straight-line spherical distances. It successfully estimates arrival times based on an average city speed of 30 km/h, providing sufficient and reliable accuracy for a backend proof-of-concept.

2. Architectural Complexity and Design Pattern Conflicts

- **Risk:** Implementing multiple structural and behavioral design patterns concurrently (e.g., Singleton, Factory, State, Strategy, and Observer) introduces the risk of tight coupling or state machine conflicts if not carefully managed.
- **Mitigation:** We established rigorous UML documentation prior to full-scale coding. Furthermore, the team dedicated Weeks 7 and 8 strictly to internal code reviews and executing continuous test cycles. This iterative feedback loop allowed us to instantly identify logical flaws and improve the code based on test case results.

3. Documentation and Code Inconsistency

- **Risk:** As the system codebase dynamically evolved through iterative testing and bug fixing, there was a high probability that the final Java

implementation would drift away from the initial system requirements and UML diagrams.

- **Mitigation:** To prevent architectural drift, the team held a dedicated synchronization meeting via Zoom in Week 10. During this meeting, all diagrams, mockups, and reports were thoroughly reviewed internally to guarantee strict consistency and completeness between the written document and the final codebase.

7. Conclusion & Reflection

In this software design project, we successfully built a comprehensive model for the system, covering everything from requirements analysis to prototype design. Through writing this report, our team came to deeply understand that UML modeling is not merely about drawing diagrams; it is a crucial bridge that translates abstract requirements into a concrete logical architecture. Our team learned how to flexibly apply design patterns (such as the Strategy Pattern and Observer Pattern) to resolve system coupling issues, and to adhere to the SOLID principles to ensure the robustness and scalability of the software architecture.

Reflecting on the entire development process, our team come to appreciate the importance of “design” and “communication.” When drawing class diagrams and sequence diagrams, even the slightest discrepancy in definitions can lead to overall logical conflicts, which has made me realize how crucial it is to establish a rigorous and consistent design document before beginning actual coding. This experience not only enhanced my technical skills but also taught me how to collaborate within a team to reach consensus. In the future, I will place greater emphasis on design details and foresight, striving to identify the optimal architectural solutions for complex software requirements.

8. References

No external references were used in the preparation of this report.